KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 02

Quality of Service (QoS) in Kubernetes

Topic: QoS Classes | CKA Weekend Course · Video 6, 3rd Sep | 2026-09-10

The Core Question

If a Pod's YAML doesn't define any CPU/RAM, and you `describe` that pod, you can't tell how much CPU/RAM it's actually allowed to consume. So: **can a pod with no resources defined eat an entire node's memory?** Yes — it's possible, and that's exactly the problem QoS classes exist to manage.

Two Types of Resources — Compressible vs Non-Compressible

- **CPU = compressible.** If a pod needs CPU and doesn't get it, it just runs on fewer CPU cycles — slower, but it keeps running.
- **Memory = non-compressible.** If a pod needs memory and doesn't get it, there's no "running slower" option — it literally cannot run with less than what it needs.

> 💡 **Why this matters**
>
> This distinction is *why* CPU and memory are handled completely differently when a pod exceeds its limit — it's not an arbitrary Kubernetes design choice, it follows directly from the physical nature of the two resources.

The 3 QoS Classes

1. BestEffort — lowest priority

- No CPU/memory mentioned in YAML at all.
- Kubernetes will "try to give its best" but **can't commit** anything.
- These are the **least priority** pods — first to be evicted under node pressure.

2. Burstable — second priority

- YAML has a `resources` block with **both** `requests` **and** `limits`, and `limits` > `requests`.
- **Request = guaranteed** — whatever you put in `requests` is given to the pod the moment it starts.
- **Limit = a threshold**, not a guarantee. Example: pod requests 64Mi memory. Under normal load it might only use 24Mi. If load increases, it can burst up to its 64Mi request comfortably, and beyond that up to a 128Mi limit if the node has spare capacity — 128Mi is the hard ceiling.

- Scheduling decisions are made based on the **request** value, not the limit.

3. Guaranteed — highest priority

- `resources.requests` == `resources.limits` for CPU **and** memory.
- These are the **highest priority** pods.

Why "Guaranteed" Is Highest Priority — Eviction Order Under Node Pressure

Picture Node A with 32Gi RAM, increasingly loaded with Guaranteed-class pods until the node comes under memory stress:

1. Kubernetes evicts **BestEffort** pods from Node A first (moves them to another node) to free up room for the Guaranteed pods the scheduler decided must run there.
2. If the node is *still* under stress after that, it starts evicting **Burstable** pods next.
3. **Guaranteed** pods are evicted last (they're the most protected class).

This is the actual mechanism behind "Guaranteed = highest priority" — it's not just a label, it's literally the last class touched during an eviction cascade.

What Happens When a Running Pod Exceeds Its Resource Boundary

Exceeds...	What happens	Pod IP	Why
Memory limit	Container gets **OOMKilled** and **restarts** — `CrashLoopBackOff` if it keeps happening, restart count increases	**Stays the same** — the container restarts, not the pod	Memory is non-compressible, no "slow down" option
CPU limit	Container is **throttled** (slowed down), never restarted	N/A — no restart happens	CPU is compressible, K8s just gives fewer cycles
Node has no room at all	**Pod gets evicted** to a different node	**Changes** — new pod, new node, new IP	Pod-level eviction, different from an in-place restart

> **CPU Throttling — the highway analogy**
>
> A highway with only 4-5 cars lets every car drive at 140-150 km/h. Put 1000 cars on the same highway and everyone has to slow down — nobody gets removed from the highway, traffic just slows. That's CPU throttling: the app keeps running, just slower, whenever it hits its CPU limit.

> ⚠️ **Interview-favorite distinction**
>
> Exceeding **memory** = restart (container-level, same pod IP). Exceeding **CPU** = throttle, never a restart. Running **out of node capacity** entirely = pod eviction (pod-level, new IP on the new node).

Commands Used

```bash
kubectl describe pod <name>     entity: shows "QoS Class:" field directly
kubectl top pods                # live CPU/memory usage per pod
cat guaranteed.yaml              # example pod with requests == limits
```

Practical Example Seen

A `guaranteed.yaml` pod (`qos-demo-ctr` container, `nginx` image) with requests == limits on both CPU and memory:

```yaml
resources:
  limits:
    memory: "128Mi"
    cpu: "100m"
  requests:
    memory: "128Mi"
    cpu: "100m"
```

...request == limit on both, so this pod is `QoS Class: Guaranteed`.

A separate `stress` pod (namespace `learning`, image `lovelearnlinux/stress:latest`) was deliberately run with `--vm-bytes 275M` against a 250Mi memory **limit** / 150Mi **request** — i.e. intentionally asking the stress tool to allocate more memory than the container's limit allows. Result: pod showed `STATUS: Error` with `RESTARTS: 2` in `kubectl get pods -o wide` — exactly the OOMKill-and-restart behavior described above.

✅ What I Practiced

- [ ] Compared `kubectl describe pod` output across a BestEffort pod, a Burstable pod, and the Guaranteed `guaranteed.yaml` pod — confirmed the `QoS Class:` field matches expectations for each.
- [ ] Ran a `stress` pod configured to deliberately exceed its memory limit and watched it hit `Error` / `CrashLoopBackOff` with an increasing restart count via `kubectl get pods -o wide`.
- [ ] Used `kubectl top pods` to watch live memory/CPU usage against requests/limits.

Merged into the master reference notes: k8s-master-notes.md §6 (Scheduling & Autoscaling) — "Quality of Service (QoS) Classes — Deep Dive" addendum.